

math_saas/utils/db.py

```
import streamlit as st
```

```
from supabase import create_client, Client
```

```
from typing import Any, Dict
```

```
# -----
```

```
# BASE CLIENT (ANON KEY ONLY)
```

```
# -----
```

```
def _base_client() -> Client:
```

```
    url = st.secrets["supabase"]["url"]
```

```
    anon = st.secrets["supabase"]["anon_key"]
```

```
    return create_client(url, anon)
```

```
# -----
```

```
# GET SUPABASE CLIENT (AUTH-AWARE)
```

```
# -----
```

```
def get_supabase() -> Client:
```

```
    """
```

```
    Returns a Supabase client.
```

```
    If access_token + refresh_token exist, restores the authenticated session.
```

```
    Otherwise returns an anonymous client safely.
```

```
    """
```

```
    client = _base_client()
```

```
    access_token = st.session_state.get("access_token")
```

```
    refresh_token = st.session_state.get("refresh_token")
```

```
# Only restore session if BOTH tokens exist
```

```
if access_token and refresh_token:
```

```
    try:
```

```
        client.auth.set_session(access_token, refresh_token)
```

```
    except Exception:
```

```
        # If tokens are invalid (expired/cleared), fall back to anon client
```

```
        pass
```

```
return client
```

```
# -----
```

```
# REQUIRE LOGGED-IN USER (UNIFIED MODEL)
```

```
# -----
```

```
def require_user() -> Dict[str, Any]:
```

```
    """
```

```
    Ensures the user is logged in.
```

```
    Accepts both dict (profiles table) and Supabase User objects.
```

```
    Returns a normalized dict.
```

```
    """
```

```
    user = st.session_state.get("user")
```

```
# Case 1: dict from login handler
```

```
if isinstance(user, dict):
```

```
    return user
```

```
# Case 2: Supabase User object (Pydantic)

if user is not None:

    meta = getattr(user, "user_metadata", {}) or {}

    email = getattr(user, "email", None)
```

```
# Normalize into dict

return {

    "id": getattr(user, "id", None),

    "email": email,

    "full_name": meta.get("full_name"),

    "name": meta.get("name"),

    "is_admin": meta.get("is_admin", False),

}
```

```
# Case 3: no user at all

st.error("You are not logged in.")

st.stop()
```

math_saas/utils/razorpay.py

```
import streamlit as st

import razorpay

def get_razorpay_keys():

    r = st.secrets.get("razorpay", {})

    key_id = r.get("key_id")

    key_secret = r.get("key_secret")
```

```

    return key_id, key_secret

def get_razorpay_client():
    key_id, key_secret = get_razorpay_keys()
    if not key_id or not key_secret:
        return None
    return razorpay.Client(auth=(key_id, key_secret))

```

math_saas/subscriptions/utils.py

```

from typing import Any, Dict

```

```

# -----

```

```

# Format currency

```

```

# -----

```

```

def format_inr(amount_paise: int) -> str:

```

```

    rupees = amount_paise / 100

```

```

    return f"₹{rupees:,.2f}"

```

```

# -----

```

```

# Safe dict getter

```

```

# -----

```

```

def safe_get(d: Dict[str, Any], key: str, default: Any = None) -> Any:

```

```

    value = d.get(key, default)

```

```

    return value if value is not None else default

```

```

# -----

```

```
# Convert plan code to readable name
# -----
def plan_name(plan_code: str) -> str:
    mapping = {
        "FREE": "Free Plan",
        "MONTHLY": "Monthly Plan",
        "ANNUAL": "Annual Plan",
    }
    return mapping.get(plan_code, plan_code)
```

Small view of Project Directory Structure

EXPLORER

✓ OPEN EDITORS

razorpay_checkout.py pages

✕ *utils.py* subscriptions

✓ MATH_SAAS



> __pycache__

> .devcontainer

> .github

> .streamlit

> admin

> api

> docs

> math-saas-venv

✓ pages

razorpay_checkout.py

razorpay_checkout.py

> services

> student

> subscriptions

> themes

> utils

__init__.py

.env

.gitignore

markdownlint.json

> OUTLINE

> TIMELINE